

TensorNet Subpackage: Usage and Coding Documentation

© Copyright 2023, 2025 Anthony D. Dutoi

This file is part of **Qode**.

Qode is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

Qode is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with **Qode**. If not, see <http://www.gnu.org/licenses/>.

1 Quick Start

1.1 Installation

Qode can be obtained from <https://github.com/adutoi/Qode>. If you have this file, you probably already did that. Next, set your python path. In a **bash** terminal:

```
1 % export PYTHONPATH=$PYTHONPATH:<.../path/to/Qode>
```

where `<.../path/to/Qode>` is replaced by the path to the cloned packaged directory (named **Qode/**, unless you changed it, and which should have **qode/** as a subdirectory). You then import the **TensorNet** subpackage of **Qode** into your python code via

```
1 from qode.math import tensornet
```

or by importing selected identifiers, as done in the examples below. **TensorNet** works at least with **Python 3.12** onward, and probably a few earlier version as well.

The minimal third-party requirement for using **TensorNet** is the installation of some package for handling the backend tensors (most basically, for example, **NumPy**) because **TensorNet** really just defines a high-level, cleanly readable syntax for tensor manipulations. It does none of its own tensor arithmetic. The most “heavy lifting” that it does is the storage of the structures of many contracted tensors (a tensor network*), and implementing a heuristic for the optimal contraction path for a given network, delegating the contractions to the backend tensor engine. [* tensor network: tensors are nodes of a graph and contractions and reductions are “multipartite edges.” See figure in introduction after this Quick Start section.]

If using **NumPy** or **TensorLy** (and maybe a few others by the time you read this) there are already interface files for the backends that allow for a seamless, get-up-and-running experience, but effort has been made to make the construction of a new interface file also extremely easy (perhaps as short as 50 lines, or less if the syntax is similar enough to standard tensor libraries

($A \neq B$, $A[i,j,k,\dots]$, etc.) to build off the interface base class (which should exist soon after writing this). Read on for more information.

1.2 Usage

The following code is imported directly from `Qode/qode/math/tensornet/tests/tutorial.py`. It is in the `tests/` directory because running it (just `python tutorial.py`) and checking the output according to the embedded comments serves as a quick (but not exhaustive) test that the subpackage is working correctly. It is named `tutorial.py` because it is heavily commented to illustrate basic usage:

```
1 # (C) Copyright 2023, 2025 Anthony D. Dutoi
2 #
3 # This file is part of Qode.
4 #
5 # Qode is free software: you can redistribute it and/or modify
6 # it under the terms of the GNU General Public License as published by
7 # the Free Software Foundation, either version 3 of the License, or
8 # (at your option) any later version.
9 #
10 # Qode is distributed in the hope that it will be useful,
11 # but WITHOUT ANY WARRANTY; without even the implied warranty of
12 # MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
13 # GNU General Public License for more details.
14 #
15 # You should have received a copy of the GNU General Public License
16 # along with Qode. If not, see <http://www.gnu.org/licenses/>.
17 #
18 # Usage: python tutorial.py [<scale>]
19 # See the message printed at the end for how to set the optional <scale>
20
21 import sys
22 import math
23 import numpy
24 import tensorly # can comment out if using numpy directly below
25 #import torch
26 #tensorly.set_backend("pytorch") # default is numpy
27 from qode.math.tensornet import np_tensor, tl_tensor, evaluate, raw,
    increment, scalar_value, ContractionError
28
29 print("""
30 Comments in the code explain the basic usage of the TensorNet subpackage.
31 If everything is working, this should print out a bunch of random numbers
32 that are < ~1e-15, a handled message exception from an intentional mistake,
33 and a single instance of a warning about extracting individual elements.
34 """)
35
36 p,q,r,s = 'pqrs' # lower the number of quotes we need to type
37
38 #####
39 ### In tensornet, backend is a tensor-local concept, but this file does not
40 ### make use of multiple backends, so we just pick one below. The factory
41 ### objects np_tensor and tl_tensor imported above act to both wrap "raw"
42 ### tensors (of the type provided by the backend we will be using) and
43 ### provide other conveniences (like the instantiation of an appropriate
```

```

44 ### zeros tensor without instantiating and wrapping in separate steps).
45 ### We set prim_tensor here to the factory that we want to use for now.
46 ### The identifiers zeros and einsum are from the raw backend, and are used
47 ### here for purposes of comparison/testing.
48
49 # use numpy directly as backend
50 #zeros, einsum, prim_tensor =      numpy.zeros,      numpy.einsum, np_tensor
51
52 # use tensorly backend (numpy or pytorch, per choices above)
53 zeros, einsum, prim_tensor = tensorly.zeros, tensorly.einsum, tl_tensor
54
55 #####
56 ### Package up some repetitive tasks (making tensors, checking accuracy)
57
58 def check(xpt, ref):
59     #      numpy
60     #return numpy.linalg.norm(xpt - ref) / numpy.linalg.norm(ref)
61     #      tensorly (BTW, what tensorly calls \|ell^2 is Frobenius)
62     return tensorly.norm(xpt - ref) / tensorly.norm(ref)
63
64 def random_tensor(*shape):
65     #      numpy
66     #return numpy.random.random(shape)
67     #      tensorly numpy (default)
68     return tensorly.tensor(numpy.random.random(shape))
69     #      tensorly pytorch
70     #return tensorly.tensor(torch.rand(*shape))
71
72 #####
73 ### Now, on with the usage tutorial ....
74
75
76
77     ### The Basics
78
79 # Some original "raw" tensors
80 A_ = random_tensor(10, 10)
81 B_ = random_tensor(10, 8)
82 C_ = random_tensor(8, 10)
83 D_ = random_tensor(8, 8)
84 U_ = random_tensor(8)
85 V_ = random_tensor(10)
86
87 # Prepare "primitive" tensors for tensornet using wrapping function for raw
88 # arrays. The original data is never modified by tensornet machinery.
89 A = prim_tensor.init(A_)
90 B = prim_tensor.init(B_)
91 C = prim_tensor.init(C_)
92 D = prim_tensor.init(D_)
93 U = prim_tensor.init(U_)
94 V = prim_tensor.init(V_)
95
96 # For our first set of tests, we will duplicate the result of the following
97 # "raw" contraction using tensornet syntax.
98 test = 2 * einsum("pr,rt,sq,ts->pq", A_, B_, C_, D_)
99

```

```

100 # - The central operation for which tensornet was designed is the building
101 #   of contraction networks. Below, the basic syntax is illustrated. It
102 #   does no computation at this point; it only logs the inputs necessary
103 #   to define the network.
104 # - Each factor connected by @ is generated by a call "()" on the input
105 #   tensor (primitives here, but can be themselves networks), whose own
106 #   arguments are either (1) a label non-integer label that specifies that
107 #   a certain index will be summed, and whose recurrence (if any) specifies
108 #   indices to be contracted with each other, or (2) an integer whose order
109 #   specifies the permutation of free indices in the contracted product.
110 # - Repeated integers specify indices to be set equal to each other (as if
111 #   contracted with a Kronecker). Such a thing is called a reduction here.
112 # - A contraction label (often a letter) can be any hashable object that
113 #   is not an integer.
114 # - An exception will be raised if a logical error occurs with the shapes,
115 #   like contracting indices of incompatible lengths.
116 # - There is no limit on the number of tensor factors and (to within machine
117 #   precision) the result will not depend in any way on how they are ordered
118 #   (not even efficiency).
119 # - Scalar multiplication is also possible in the same string, but the *
120 #   operator should be used for scalars.
121 # Here is now the tensornet syntax and the check of its accuracy:
122 ABCD1 = 2 * A(0,p) @ B(p,q) @ C(r,1) @ D(q,r)
123 print("relative error in check 1:", check(raw(ABCD1), test))
124
125 # This can also be done in stages as follows
126 AB2    = A(0,p) @ B(p,1)
127 CD2    = C(p,1) @ D(0,p)
128 ABCD2  = AB2(0,p) @ CD2(p,1)
129 ABCD2 *= 2
130 print("relative error in check 2:", check(raw(ABCD2), test))
131
132 # The point of the next set of definitions is that they are functionally
133 # equivalent to the ones above, except in one subtle detail.
134 AB3    = (A(0,p) @ B(p,1))()
135 CD3    = (C(p,1) @ D(0,p))()
136 ABCD3  = (AB3(0,p) @ CD3(p,1))()
137 ABCD3 *= 2
138 print("relative error in check 3:", check(raw(ABCD3), test))
139 # The existence of the syntax using the intermediate "()" is mostly just a
140 # reassurance for the paranoid, but it is important to state why it exists.
141 # We needed 4 distinct indices in test 1 because the expression is still
142 # "open" after contracting A with B, and using p again in this expression
143 # (as we do in tests 2 and 3) would be interpreted as the same index (a
144 # 4-index contraction). Consider that one would then also get an erroneous
145 # result from
146 AB4 = 2 * A(0,p) @ B(p,1)
147 try:
148     ABCD4 = AB4 @ C(p,1) @ D(0,p)
149 except ContractionError as err:
150     print(str(err) + "\n")
151 # because, according to python syntax, it is equivalent to using p four
152 # times in the expression just discussed and setting other indices equal
153 # to each other. In fact because these dimensions are not even of the
154 # same length, we do not just get a wrong answer, we get an exception,
155 # which is ignored here (but the message is printed as an example). However,

```

```

156 # the following gives the correct result for the second line
157 ABCD4 = AB4(0,p) @ (C(p,1) @ D(0,p))(p,1)
158 print("relative error in check 4:", check(raw(ABCD4), test))
159 # The reason is that we have a concept of "open" and "closed" expressions
160 # (more technically just expressions (open) and expressions that have been
161 # resolved to a tensor network (closed)). The repeated usage of @ and *
162 # (for scalars) builds open expressions, but applying any other operation
163 # to an open expression closes it before anything else happens. Once an
164 # expression has been resolved to a network, it has no memory of the letters
165 # that were once used to describe its connectivity, and it cannot even enter
166 # into another expression until a "(" has been performed on it. Similarly
167 # acting a ")" on an expression closes it automatically before the action
168 # of the call is applied. Though typical uses would not be likely to cause
169 # one to accidentally extend an open expression, one can close it manually
170 # without needing to specify indices by using an empty call (which is
171 # equivalent to just using call with the arguments (0,1,2,...,<n>-1), where
172 # <n> is the dimensionality of the result of the expression.
173
174 # In the tests above, we have repeatedly used the raw() function. When
175 # acting on a tensornet tensor, this triggers evaluation and it returns the
176 # result as a "raw" backend tensor to the user. There is a similar
177 # evaluate() function with returns the result as a tensornet "primitive"
178 # tensor that can be used in subsequent tensornet expressions. More on that
179 # below.
180
181 # Below illustrates that +, - and * all work on tensors networks, and the
182 # call notation "(" can also be used just to permute indices. The result
183 # of a scalar multiplication (which is fast) is another tensor network whose
184 # internal scalar factor is modified. Similarly, the result of the
185 # subtraction is an abstracted structure that keeps track of the operation
186 # but performs no computation until evaluate() or raw() is used.
187 ABCD_p = 3*ABCD1 - 3*ABCD1(1,0)
188 test = 3*test - 6*einsum("pr,rt,sq,ts->qp", A_, B_, C_, D_)
189 print("relative error in check 5:", check(raw(ABCD_p), test))
190
191 # This illustrates that there are not many restrictions on the order of the
192 # arguments or the placement of the indices. The redundant "1" means that
193 # the 1 index of the contracted product is resolved by setting these two
194 # equal to each other. Also single as well as multiple (not just double)
195 # occurrences of contraction labels are allowed. Note that the same index
196 # can occur multiply on any given tensor, and that outer products (A shares
197 # no indices with others) are allowed. Finally, if a scalar will be buried
198 # in the middle of a contraction string, it is best to use @ on both sides
199 # or else you have to remember to use * only for the first one because of
200 # the order in which python operators bind.
201 ABCD3 = A(q,0) @ B(1,p) @ 3 @ C(p,1) @ D(p,p)
202 test = 3 * einsum("qr,sp,ps,pp->rs", A_, B_, C_, D_)
203 print("relative error in check 6:", check(raw(ABCD3), test))
204
205 # The real utility of tensornet is for something like this. E represents
206 # 4-index tensor, but its contractions with vectors can be done faster if
207 # done internally as contractions with the factors. The scalar_value()
208 # function extracts the scalar-typed value from a 0-index tensor (like
209 # ndarray.item() in numpy).
210 E = A(0,1) @ D(2,3)
211 VVEUU = scalar_value(E(p,q,r,s) @ V(p) @ V(q) @ U(r) @ U(s))

```

```

212 test = einsum("pq,rs,p,q,r,s->", A_, D_, V_, V_, U_, U_).item()
213 print("relative error in check 7:", (VVEUU - test) / test)
214
215 # You can take elements of a primitive tensor, ...
216 print("relative error in check 8:", (A[0,0] - A_[0,0]) / A_[0,0])
217 # ... or of a tensor network. For a network, this will reevaluate
218 # everything for every element requested, so it warns the user the first
219 # time about this being inefficient (maybe use evaluate() on a block and
220 # then take elements instead?). To be clear, tensornet is smart enough not
221 # evaluate the whole tensor to obtain a single element; it first restricts
222 # the relevant indices in the primitives before contraction is done. So, in
223 # this particular case, it actually is not inefficient.
224 print("relative error in check 9:",
225       (E[0,0,0,0] - A_[0,0]*D_[0,0]) / (A_[0,0]*D_[0,0]))
226 # Note that it does not warn you a second time.
227 print("relative error in check 10:",
228       (E[1,1,1,1] - A_[1,1]*D_[1,1]) / (A_[1,1]*D_[1,1]))
229
230 # Slicing also works, ...
231 print("relative error in check 11:", check(raw(A[0, 1:5]), A_[0, 1:5]))
232 # ... within contractions ...
233 AC = A[:, :8](0,p) @ C(p,1)
234 test = einsum("pq,qr->pr", A[:, :8], C_)
235 print("relative error in check 12:", check(raw(AC), test))
236 # ... and on tensor networks (where again it first acts to slice the
237 # primitives in the network before evaluation is done.
238 test = einsum("p,q->pq", A[:, 0], D_[:, 1])
239 print("relative error in check 13:", check(raw(E[:, 0, :, 1]), test))
240
241
242
243     ### Some Finer Points
244
245 # One thing that is good to be aware of is that, if one contracts a sum
246 # (such as F below), the contractions of the terms are always done first and
247 # then added (most likely desired and far simpler dispatching algorithm) ...
248 F = A + 5*B(0,p)@C(p,1)
249 test = einsum("p,pq,q->", V_, A_, V_) + \
250       5 * einsum("p,pq,qr,r->", V_, B_, C_, V_)
251 print("relative error in check 14:", check(raw(V(p) @ F(p,q) @ V(q)), test))
252
253 # ... but the user has control over this by forcing evaluation of the
254 # intermediate first, if desired. The evaluate() function performs all
255 # internal contractions and index reductions and returns a primitive (not a
256 # raw) tensor.
257 print("relative error in check 15:",
258       check(raw(V(p) @ evaluate(F)(p,q) @ V(q)), test))
259
260 # Should one need to loop over terms to build a sum, an empty accumulator
261 # can be started as so:
262 F = prim_tensor.zeros()
263 for term in [A, 5*B(0,p)@C(p,1)]: # pretend generated by some algorithm
264     F += term
265 print("relative error in check 16:", check(raw(V(p) @ F(p,q) @ V(q)), test))
266 # In this example F takes its shape from the first term and the checks that
267 # the shapes of subsequent terms match. This means that attempting to

```

```

268 # evaluate F before the summation started would give an exception because it
269 # has no valid shape, but a shape argument can be given and an explicit
270 # tensors of zeros would be generated by the backend on demand.
271
272 # Here is another way to do this, if you know you want a concrete evaluation
273 # immediately. This does not yet save memory with an in-place build, but
274 # the point is to leave the door open for that to be implemented later
275 # (presently just a thin wrapper around +=raw(...)).
276 F_ = zeros(A.shape)
277 for term in [A, 5*B(0,p)@C(p,1)]:
278     increment(F_, term)
279 print("relative error in check 17:",
280       check(einsum("pq,p,q->", F_, V_, V_), test)),
281
282 # Oops, looks like I explained this material twice. I should pick on
283 # and probably put it in this location because it is too much at the
284 # beginning.
285 # And now a comment on a subtlty of the @ operator. By necessity (since
286 # there is no trigger to tell it to stop), it generates "incomplete"
287 # contractions. Consider "ABCD" from above. Done in the way below, the
288 # contraction labels q and r must be shared across two lines. This is a
289 # potential danger since one might not realize a letter has been shared if
290 # lines like these are farther apart. Whereas there might actually be a
291 # niche use case for this behavior it is mostly just a dark corner.
292 ABC4 = A(0,p) @ B(p,q) @ C(r,1)
293 ABCD4 = 2 * ABC4 @ D(q,r)
294 test = 2 * einsum("pr,rt,sq,ts->pq", A_, B_, C_, D_)
295 print("relative error in check 18:", check(raw(ABCD4), test))
296 # However, under normal circumstances, one would not write code in this way,
297 # so the danger is limited. If one really means that, in ABC4, q and r
298 # should be summed over as individual indices, leaveing only two free
299 # indices in the product, then future contractions should be written more
300 # like the following (note the call with indices attached to ABC4), where we
301 # have to choose something other than D just for dimension matching.
302 ABCB1 = 2 * ABC4(p,0) @ B(p,1)
303 test = 2 * einsum("pq,qr,st,pu->tu", A_, B_, C_, B_)
304 print("relative error in check 19:", check(raw(ABCB1), test))
305 # And as a one-liner, we could have
306 ABCB2 = 2 * (A(0,p) @ B(p,q) @ C(r,1))(p,0) @ B(p,1)
307 print("relative error in check 20:", check(raw(ABCB2), test))
308 # And, just for clarity, this is of course the same (in every way) as
309 ABCB3 = 2 * A(s,p) @ B(p,q) @ C(r,0) @ B(s,1)
310 print("relative error in check 21:", check(raw(ABCB3), test))
311
312
313
314     ### About Efficiency
315
316 # Here is a quick test that shows this code can be smarter than einsum in
317 # dispatatching contractions.
318 # Setup ...
319 factor = 1
320 if len(sys.argv)==2:
321     factor = float(sys.argv[1])
322 dim = math.floor(factor * 10)
323 M_ = random_tensor(dim, dim, dim, dim)

```



```

324 M = prim_tensor.init(M_)
325 T_ = random_tensor(dim, dim)
326 T = prim_tensor.init(T_)
327 # ... and test:
328 TTMTT = raw(M(p,q,r,s) @ T(p,0) @ T(q,1) @ T(r,2) @ T(s,3))
329 print("Tensornet done with 4-index transformation. Waiting on einsum ... ")
330 # See note about opt_einsum in to-do (default in pytorch and accessible with
331 # numpy, also via tensorly)
332 test = einsum("pqrs,pw,qx,ry,sz->wxyz", M_, T_, T_, T_, T_)
333 print("... einsum done.")
334 print("relative error in check 22:", check(TTMTT, test))
335 print("If the timing difference was not dramatic enough, "
336       "run the script with a float argument >~1")
337 print("to scale the dimensions. But do not go nuts. "
338       "The einsum call scales with the 8th power.")

```

Here are some more useful sentences about the idea of incomplete and not-yet-resolved expressions.

it would be incorrect to assume that a complete and valid tensor has been described after every use of the @ operator. Therefore, it is not safe to assume that the use of @ results in something that is a tensor in every way (eg, it may not even have a valid shape, due to missing free indices). since the = operator in python only means "bind" (as opposed to, say, C++), there is no way for the user to indicate when a tensor description is "done". (correction, now use `__call__()`) The dark corner here is that the result of several @ operations remembers the names of its contraction indices and if that result is bound to a variable, it could be @-ed with more tensors, inadvertently using the same letters, leading to unexpected results. This is probably not a big deal, however, because this usage pattern is also unorthodox, since, after binding the result of an @ string to a name, that object should itself be subjected to the call operation () referencing the indices of the finished product (with no memory of labels used for internal contractions) before being contracted with other tensors.

2 More Stuff

2.1 Theory

Our goal is to evaluate expressions of the following form for the elements of tensor **Z**

$$Z_{pqr} = \sum_{stuvwxy} A_{qpst} B_{stuv} C_{uqw} D_{uxryy} E_{wx} \quad (1)$$

This is probably not a realistic form for any practical problem, but it illustrates a number of features: Index v is summed but occurs only once, whereas index u is shared between three tensors, index y is trace-like on one tensor only, and one of the uncontracted (free) indices occurs twice on the right-hand side; the others are more "normal," but, at that, **A** and **B** are connected by two such indices. For lack of a better word, we will call the multiple appearance of free indices a "reduction", as it reduces the number of tensor axes, and a subsequent sum over it would be a contraction.

The structure of the right-hand side of Eq. 1 can be represented as a hypergraph, as shown in Fig. 1, where the nodes are the tensors, and the hyperedges are the summations over indices (henceforth, just "graph" and "edges"). The index positions of the associated tensors that are connected can be thought of as a properties of the edges (*e.g.*, there is an edge that connects

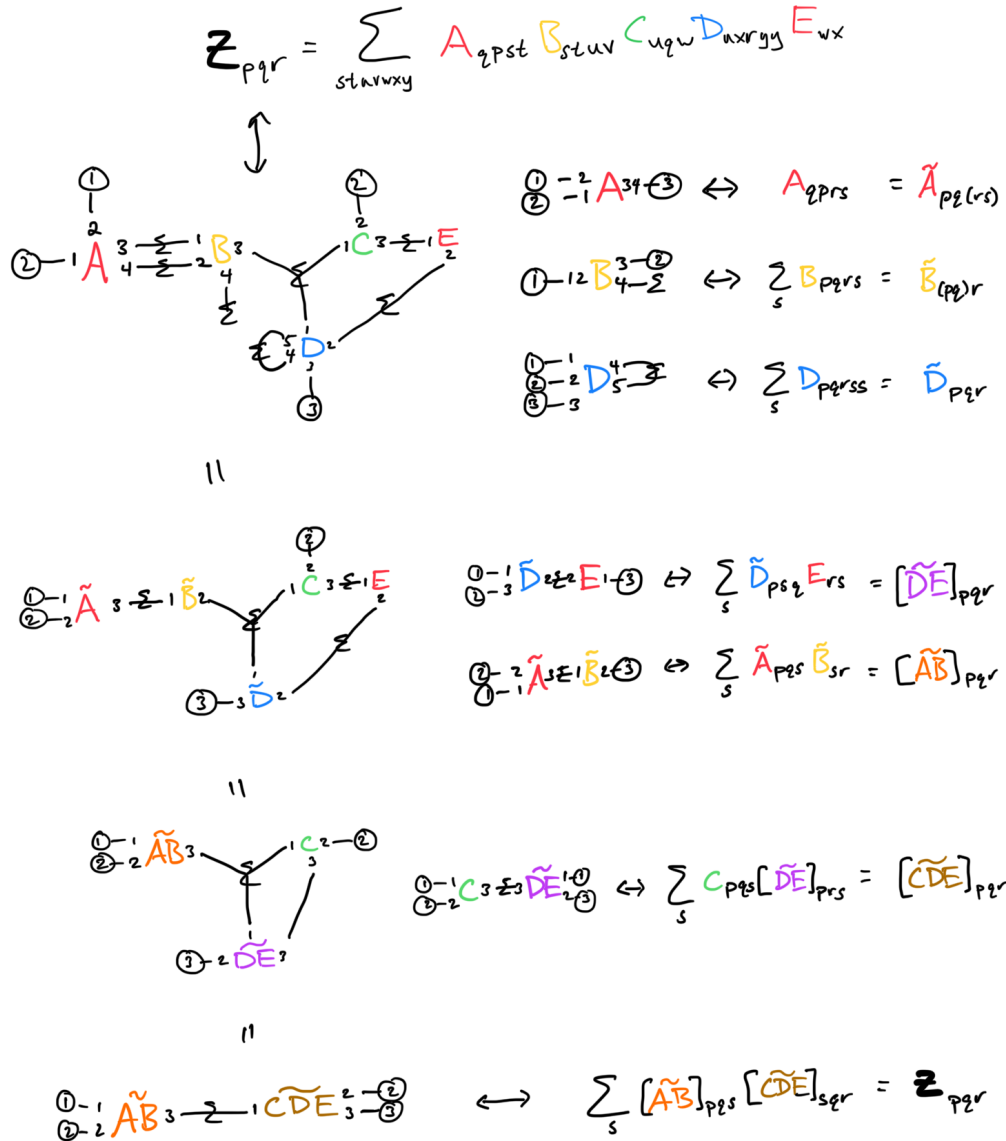


Figure 1: Depiction of the definition and hypothetical evaluation flow of a horrible, unrealistic tensor network, represented as a hypergraph. Evaluation of sub-graphs proceeds down the page until explicit evaluation is complete. The algebraic definitions of the sub-graphs used are also given. The evaluation flow is purely hypothetical, only to illustrate the kinds of steps that can be taken, rather than indicating an optimal sequence. I should have built $\tilde{\mathbf{A}}\tilde{\mathbf{B}}\mathbf{C}$ in the last step instead of $\tilde{\mathbf{C}}\mathbf{D}\mathbf{E}$ to illustrate a reduction occurring before the final evaluation?

the 3rd index of $\mathbf{A}$ to the 1st index of $\mathbf{B}$, where counting in this representation starts at 1). The free indices are thought of as simply being properties of the nodes that are not technically part of the graph. Nevertheless, we have a special notation to keep track of these. The ordering of the free indices in the product is given by circled numbers connecting to the corresponding index positions of the inputs. Should we have an index that is simply summed, we just draw a line from its position to a sum sign, but a dummy vector could formally be introduced as a node here. An important feature is that the graphs need no index “letters” for their description;

only non-arbitrary identifiers are necessary to internally represent a tensor network, once it is resolved from the user input via the front end syntax.

Given a fully resolved network, it will eventually be required to resolve it. The sequence given in Fig. 1 is not meant to be optimal. It is meant to illustrate the kinds of steps that can be taken. `TensorNet` does contain a heuristic to try to identify the most likely optimal path, and it works very well in simple situations. More importantly however, this heuristic is separable from the rest of the code (under a couple of assumptions addressed later). Swapping this out (and providing an interface to any options) would be a developer task, as opposed to a user task (such as providing an interface to the backend is a user task), but it should be pretty straightforward to make use of more advanced algorithms as long as the storage paradigms for the network information is translatable (see Code Structure). For purposes of illustration, however, we will comment that the first steps are obviously correct. The first step is obviously correct, however. For what can only result in future reduction of arithmetic and memory usage, **D** can be contracted to a smaller quantity without changing structure. Similarly, though it does not result in any reduction of problem size, the structure of the diagram can be simplified by considering the two contractions connecting **A** and **B** to run over a single super-index; it is clearly most efficient to perform those contractions in a single step, when appropriate. Note that if the diagram had not contained a loop structure, the contraction over the triply shared index could have been done in a single step.

Regardless of the actual contraction path decided upon, or even how it is decided upon (using the heuristic as of the time of writing, or something more advanced that is patched in), once the sequence of steps is determined, `TensorNet` delegates a series of tensor contractions to the backend engine. The quantities received by the backend are always of its native type. Under the most normal circumstances (no hyperedges), it will receive two at a time, along with information about what indices to contract.

2.2 About Backends

The concept of a backend is local to the tensors. Rather than have a global setting, any two tensors with the same backend may enter into `tensor` expressions with each other. The long way of doing this for a `numpy` backend would be as below. The default is not to make a copy of the original tensor data, but simply to wrap it. As long as the user has no intention of changing the original data (`tensor` will not either), it is more efficient, but if you are getting weird results and want to be sure, or you just know you cannot promise not to modify the original, then use the `copy_data` flag (deprecated; can duplicate with `data_wrap` function). The `np_tensor` function used above (and provided for convenience), is equivalent to:

```
1 from qode.math.tensor import primitive_tensor_factory, numpy_backend
2 np_tensor_ = primitive_tensor_factory(numpy_backend.functions)
```

One can see now that providing a substitute for `numpy_backend` above is all one needs for a custom backend. This should be a module that has the following functions defined in it.

Here is the code for `numpy_backend`, which one can see just provides a set of functions that work with “raw” `numpy` tensors. On the to-do list is to make a base class that one can build off of, and also to recognize that the majority of the code in the `contract` function is just to translate the specification to `einsum` notation. This can be separated off as an isolated, reusable utility (along with a similar one for `tensordot` notation (which needs to raise an exception for more than two tensors at a time (using the native `tensor` dispatching, this assumption is obeyed)).

```

1 # (C) Copyright 2023, 2025 Anthony D. Dutoi
2 #
3 # This file is part of Qode.
4 #
5 # Qode is free software: you can redistribute it and/or modify
6 # it under the terms of the GNU General Public License as published by
7 # the Free Software Foundation, either version 3 of the License, or
8 # (at your option) any later version.
9 #
10 # Qode is distributed in the hope that it will be useful,
11 # but WITHOUT ANY WARRANTY; without even the implied warranty of
12 # MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
13 # GNU General Public License for more details.
14 #
15 # You should have received a copy of the GNU General Public License
16 # along with Qode. If not, see <http://www.gnu.org/licenses/>.
17 #
18 import numpy
19
20 # Need to place these in an object (a class is an object) because modules
21 # cannot be pickled. On a similar note, they need to be in a class and not
22 # a class instance because instances that are the same upon pickling may be
23 # unpickled as distinct objects, which interferes with checking if two
24 # backends are the same (best to used "is" for absolute assurance that they
25 # are exactly the same). That said, since modules are not pickled
26 # (referenced by fully qualified path and loaded only once), we can check
27 # that two backends are the same using the "is" operator between the class
28 # defined here.
29
30 class functions(object):
31     name = "numpy"
32     @staticmethod
33     def scalar_value(tensor):
34         return tensor.item()
35     @staticmethod
36     def scalar_tensor(scalar):
37         return numpy.array(scalar)
38     @staticmethod
39     def shape(tensor):
40         return tensor.shape
41     @staticmethod
42     def zeros(shape):
43         return numpy.zeros(shape)
44     @staticmethod
45     def increment(tensor, delta):
46         tensor += delta
47         return
48     @staticmethod
49     def mult(scalar, tensor):
50         result = scalar * tensor
51         if len(tensor.shape)==0:
52             result = functions.scalar_tensor(result)
53         return result
54     @staticmethod
55     def subscript(tensor, indices):
56         return tensor[indices]

```

```

57 @staticmethod
58 def str(tensor):
59     return str(tensor)
60 @staticmethod
61 def contract(*tensor_factors):
62     #####
63     # args = []
64     # for factor in tensor_factors:
65     #     try:
66     #         tensor,*indices = factor
67     #     except:
68     #         args += [factor]
69     #     else:
70     #         args += [(id(tensor),*indices)]
71     # print("backend.contract called with", *args)
72     #####
73     def letters(excluded):
74         i = 0
75         def next_letter():
76             nonlocal i
77             candidate = 'abcdefghijklmnopqrstuvwxyz'[i]
78             i += 1
79             if candidate in excluded:
80                 return next_letter()
81             else:
82                 return candidate
83         return next_letter
84     tensors = []
85     index_strings = []
86     all_indices = set()
87     max_int = -1
88     scalar = 1
89     for factor in tensor_factors:
90         try:
91             tens, *indices = factor
92         except:
93             scalar *= factor
94         else:
95             all_indices |= set(indices)
96             tensors += [tens]
97             index_strings += [indices]
98             for index in indices:
99                 if isinstance(index,int):
100                     if index>max_int:
101                         max_int = index
102     free_indices = []
103     if max_int>=0:
104         next_letter = letters(all_indices)
105         free_indices = [next_letter() for _ in range(max_int+1)]
106     instructions = []
107     for indices in index_strings:
108         for i in range(len(indices)):
109             if isinstance(indices[i],int):
110                 indices[i] = free_indices[indices[i]]
111             instructions += [".".join(indices)]
112     instructions = ",".join(instructions)

```

```

113     instructions += "->"
114     instructions += "".join(free_indices)
115     # print("einsum called with", instructions)
116     value = scalar * numpy.einsum(instructions, *tensors)
117     # print("value id", id(value))
118     return value

```

Here are some outdated notes that might have useful phrases in them.

```

1 #
2 def copy_data(tensor):
3     Returns a tensor referencing independent numerical data that is a copy of
4     that of tensor
5     for numpy this just returns numpy.array(tensor)
6 #
7 def scalar_value(tensor):
8     Assuming the tensor has no indices left, return the internal value as a
9     scalar type (like float)
10    For numpy, this just returns tensor.item()
11 #
12 def scalar_tensor(scalar):
13     Return the scalar in the data type of a 0-index tensor
14     For numpy, this just returns numpy.array(scalar)
15 #
16 def shape(tensor):
17     Give the shape (dimensions) of tensor as a tuple of integers
18     For numpy, this just returns tensor.shape
19 #
20 def contract(*tensor_factors):
21     The arguments of this function are exactly of the same format as the
22     tensornet contract()
23     function. The difference is that it is likely to be called with a much
24     smaller number
25     of arguments, as the tensornet machiney grinds down the contraction
26     network stepwise.
27     A user is free to make this as general or as limited as necessary. For
28     example, if only
29     pairwise contractions of indices on two distinct tensors is every
30     necessary (by-and-large
31     the dominant use case), then this routine will only every be called as
32     such. The implementor
33     can just raise a NotImplementedError if a different use case is ever
34     given.
35     For numpy, the general use case can be mapped onto the arguments of a
36     valid einsum call.

```

PS - In this directory, you see a `dummy_tensor` module, which is supported by the `dummy_backend` module provided by tensornet. This does not do any math but just prints out the contractions that are being performed. Originally used to debug tensornet, it might come in handy for users to verify that their code is being executed as expected (and possibly find problems with the contraction-order heuristic).

PPS - About `scalar_tensor` and `scalar_value`. The idea is that many backends should and do have two different ways of thinking about scalars. There are those that are of some more basic non-tensor data type (like `float`) and then there are 0-dimensional tensors. These represent mathematically indistinguishable concepts (as distinct from say a $1 \times 1 \times \dots \times 1$ tensor). But one (the 0-dimensional tensor) is mutable and the other is not. This makes a

difference inside of tensornet in the efficient evaluation of sums which assume that the result tensor where the result is accumulated is mutable, but some functions (einsum!) are not careful and return a scalar when I think it would be more appropriate to return a 0-dimensional tensor. This creates a silent but deadly bug. I need to either insist that the user backends preserve tensors when “appropriate” or find some way to force the issue in the background. What to do if a library does not even have a notion of a 0-dimensional tensor. I should work from that assumption.

2.3 Assumptions About Sub-graph Decomposition

Here I should expose some assumptions about how graphs are to be evaluated stepwise. Any heuristic that obeys these assumptions should be straightforwardly swappable with what is presently in use. Any backend that can handle the primitive steps required is usable (and any backend can be made useable, read on).

I assume firstly that all of the nodes are explicit tensors that do not consist of sums of other tensors (ie, unless those can be, and are, cast as contractions, and are therefore just subgraphs). What cannot be cast in this form is a sum of tensors where the terms are themselves graphs of different structures. My code enforces this at the top level by turning any networks that contain sums into sums of networks. The networks are all evaluate first, and the sum is taken at the highest level. Of course, the user could just evaluate the sum themselves and provide it as a primitive input, or stack it and make it look like another tensor, but we don’t automatically handle it.

For a given network, we assume that each step will excute either one contraction group or one reduction group. A contraction group consists of all contractions of the same structure between the same nodes (ie, the only difference is the positions of the indices connected). A path that does something more complicated than this will require something more substantial than just a translator to use a different heuristic.

With respect to the backend, the backend should expect then to get such a step as just described. As long as there are no hyperedges (everything is a true edge), then it will never get more than two explicit tensors at a time, though it may be asked to contract more than one index at a time. The latter capability is ubiquitous, and the widely available einsum will handle the former as well. However, if the backend can really only do two tensors at a time, the user will have to implement the arithmetic to turn a contraction of three tensors into two contractions of two tensors (for example).

3 Code Structure

We should start with some overview of the file/directory structure.

Give an outline of `tensor_base`.

3.1 Original introduction to `tensors.py`

We have 4 different types of top-level tensors

```
contraction_expression
tensor_sum
tensor_network
```

`primitive_tensor`

All of these are built off of `tensor_base`. The latter three are built off of `resolved_tensor` (which is based on `tensor_base`) indicating that they are completely well-formed.

`contraction_expression` behaves like a tensor so long as it is well formed, but it could be incomplete. The basic problem is that, when connecting a bunch of tensors with `@`, it is not possible to automatically discern when the string is finished (ie, when all intended target free indices are present). Only the user can decide when to "close" the string, but we don't want to make the user do this explicitly. Therefore, we allow this incomplete class, whose `*resolution*` is a `tensor_network` (it always resolves to a `tensor_network`). This is triggered by taking another action on it that is not `@` or `*`. A `tensor_sum` can only contain types `primitive_tensor` and `tensor_network`. An attempt to include another `tensor_sum` will cause it to be expanded out as a single sum (and `contraction_expression` will be automatically resolved to `tensor_network`). A `tensor_network` can only contain type `primitive_tensor`, and including one network (or `contraction_expression`) into another network causes explicit expansion into a single network. Taking this together, this means that a `tensor_network` cannot contain a `tensor_sum`. An attempt to put a `tensor_sum` into a `tensor_network` will result in the hierarchy being swapped to a sum of networks. The reason for this is purely practical. At the moment, I cannot think of a way of finding the best path to evaluate a network that contains sums.

3.2 Original introduction to `tensor_network` in `tensors.py`

Barebones theory (written much later after a forensic debug battle). A `tensor_network` object contains two fundamental pieces of information (and other incidental info). One is a list of contractions. Each contraction itself is a list of two-tuples; each two-tuple identifies a tensor and the index of that tensor involved in the contraction. So if a contraction contains two two-tuples, then two indices are contracted with one another, but there might be more for unusual contractions. The ordering of the list of contractions is irrelevant, as is the ordering of the list of two-tuples that defines a given contraction. The other piece of information is a list of free indices of the result, which correspond to uncontracted indices of the tensors in the network. This list is ordered, and each free index is itself a list of two-tuples with the same tensor-index structure as the two-tuples that define a contraction. In the most usual case, each such list corresponding to a free index will be of length one, but if there is more than one tensor index that corresponds to a single free index it is because those indices are set equal to each other and reduced to a single free index.

3.3 Original introduction to `primitive_tensor_factory` in `__init__.py`

Philosophy: A backend (in the sense of the value of the argument `backend` below) is a class that contains static methods that perform basic operations on objects that represent tensors of a specific type. These will henceforth be called backend tensors. Tensornet then uses the backend functions to manipulate the backend tensors according to what is encoded by its high-level syntax. Objects of the class defined below are factories for the primitive tensornet tensors which each wrap a backend tensor (and never modifies it). An important point is that a given backend can have multiple factories depending on how what a user is doing.

The creation of a tensornet primitive tensor should only take place via a factory (which is why that class name is "hidden" here with the underscore convention) because this is the

mechanism by which a backend tensor is associated with its backend functions and the utility for contraction creation. In the most basic case, this creation is done by the `.init()` method (not `__init__`, which creates the factory). The method `.zeros()` is a convenience function for creation of a tensor net primitive storing a new backend tensor (via a backend method) so the user need not create such a zero tensor first and then "wrap" it (`.scalar_tensor()` is similar and was added for completion, but commenting it out essentially documents that we have not yet found a use case ... the necessity for the backend to be able to create such a thing however has to do with obeying expectations about type uniformity).

Finally we come to the arguments `data_wrap` and `data_unwrap`, the method `.data()`, and the idea of multiple factories for the same backend. It is important that tensor net works only with a notion of backend tensors and the functions it is given for them without knowing anything further about their properties. But the user may be working with some abstracted backend tensors. Although they contain, or somehow represent the existence of, a specific concrete tensor (eg, numpy, tensorly, pytorch), they do not have the usual expected behavior (e.g., `[]` might not even get an element). We will call such things meta tensors, it might be very inconvenient for the user to make their in-house meta tensor behave exactly like a concrete tensor, right down to letting a `numpy.ndarray` (etc) be instantiated from it. It will also be very inconvenient if the user has to always use the unbound function `raw()` and then afterward extract the concrete data for cooperation with other linear algebra libraries. The converse is also true that it will be annoyingly repetitive to always create the meta tensor from a concrete tensor from elsewhere, and then pass that to the factory. So, therefore, a factory can be created that takes the functions `data_wrap` and `data_unwrap`. Although their actions are not constrained as such, the idea is that `data_wrap` is a function that takes a concrete tensor and returns an object of the meta tensor type with which backend works, and `data_unwrap` does the opposite, extracting and returning (by reference) the concrete tensor represented (eg, contained within) the meta tensor. The convenience function `.data()` called on a tensor net tensor causes evaluation of the (potential?) network (sum?) and returns directly the concrete tensor associated with the result. By default, `data_wrap` and `data_unwrap` are just pass-throughs, which means that `.data()` is just an alias for the unbound function `raw()`, which forces evaluation and returns a backend tensor (which is typically already a concrete tensor type). Since this automatic wrapping and unwrapping might not always be the desired behavior, more than one factory can be defined, per need.

Addendum 1: There is no requirement that a meta tensor is not itself also a concrete type, and that this concrete type might be initialized from multiple other concrete types, which might not even be different than its own type. So if we interpret the concrete and meta layers to both be tensorly, where the meta layer can be initialized also from numpy or pytorch (etc), then there it is a perfectly clean for the user to provide as `data_wrap` a function that makes a tensorly tensor from some input which may or may not be of tensorly type), and to leave `data_unwrap` as the default pass-through.

Addendum2: One question that can be asked about the design is: why am I giving in a separated backend with functions that perform manipulations on a tensor, rather than just insisting that backend tensors have certain properties, like `+=` (which would also make the insides of tensor net prettier). The answer is that one might have enough serious work to do in the abstracted/meta backend already, without having to define yet more operations that make the thing look almost like a tensor, but not completely. One can imagine various inheritance and virtual method schemes, but one is always going to end up in one of two places: 1. Forcing the user to make a new class for their backend tensors (and then maybe always, even when

using "normal" tensors), or 2. automating the creation of the backend tensors with the expected operators in terms of some other/divorced set of instructions. In case 2, we are back where we started from the user point of view, though the insides of tensornet look nicer (for the price of another file/class and layer of abstraction).

Addendum 3: The `__init__()` method used to take a `copy_data` argument which meant that the factory should copy the provided primitive before storing it as an argument for further use. Probably unusual, but a user might do this if they intended on modifying that primitive for some other use while it was still in use by the tensornet code (you never know ...). This can now be done by the user with the `data_wrap` option.

3.4 About the Contraction-path Heuristic

In the event that we would want to make a heuristic that can handle embedded sums (or get one from elsewhere), here is what would need to change. The `resolve()` function would no longer invert the hierarchy of sums and networks when a sum is to be in a network, but it would simply allow it. The old heuristic could then be recovered by parsing the completed network and creating a list of networks, each of which could even have different structure, depending on what was in the sum(s) (and multiple sums would give a tensor-product like structure to the list). Then, once the networks in the list are evaluated, they are summed. However, a better algorithm might have a better idea of the order of operations to do.

4 To Do

General and highest priority:

- make sure tensornet can be imported without needing anything more than numpy in the environment. else document
- make sure that the following assumption is right: in the event of a trivial network (contains a single primitive tensor) and a scalar of 1, `evaluate()` gives a copy of the internal data (not the original) ... or maybe it is not possible to have such a network? The user can't build it, and there is no mechanism to make one.
- implement `__str__` for all tensors. as rich as possible without being uselessly verbose
- in the document above, (there are a couple places where) the concept of the distinction between a scalar and a scalar tensor appears. have a look at these and decide what to do. really want a solution that works without burdening the user with thinking about it and works even if the backend makes no such distinction (does not contain a notion of a mutable 0-dimensional tensor)
- see very end of `opt_einsum` section below for another to-do item
- look into typing (generally) and protocols (for backend interfaces): <https://typing.python.org/en/latest/index.html> <https://realpython.com/python-protocol/>

Here is a list of backend improvements (as opposed to things that are more like bugs above. If that works out right, then we should be able to get rid of `scalar_tensor` function, but we should keep `scalar_value` because a user may want it and we know that there are backends that do have a distinction.)

- have a backend base class that is purely abstract as a form of documentation, which is also sucked into this document.
- backend base class with more things implemented for “normal” backends
 - `scalar_value` → `pass`
 - `increment` → `+=`
 - `mult` → `*`
 - `subscript` → `[]`
 - `shape` → `.shape`
 - `str` → `str`
 - `field` → `float`
 - `raw_check` → `pass`
 - ↑ See below for last two ↓
- allow variable `field` for scalars (could have a name?) ... should be function that takes 1 and returns the appropriate multiplicative identity. And it should allow multiplication and be able to multiply a tensor. As a special case, i would like the scalar to be able to contain a name so that it works nicely with `test_dummy.py` in the last line.
- In addition to letting a backend check that a tensor is a valid input for a primitive `raw_check`, we should let the backend check that only compatible indices are being contracted. ie, we may want something richer than just “shape” and length (only those of the same fragment, or only `occ` with `occ`)
- Currently, if a user wants to take the norm of a tensor `T`, they have to writes something like `norm(raw(T))`. This is kind of ugly and inconvenient, but I also do not want to have a bunch of pre-defined tensor functions that are either supported or not. The point of allowing different backends is that they have different strengths, and that may mean different supported utilities. So I want to implement some system where a user can add custom functions to the backend that `tensornet` knows how to call. This might be something like using `__getattr__` on an object of `primitive_tensor_factory` class. Or maybe users should just wrap their utilities, like: `norm = lambda T: numpy.linalg.norm(raw(T))`

4.1 Questions about using `opt_einsum`

<https://joss.theoj.org/papers/10.21105/joss.00753>

Clearly there are advantages to the fact that the authors of `opt_einsum` have given much more thought to their heuristics, but it is not immediately available for all backends, and the fact that my heuristics live above the backend and need to know nothing about the hardware to at least make a *good* guess (maybe not the best) still makes it useful. That said, on further study, we might find that `opt_einsum` is just as easily put on top of other backends as is my own code (eg, it already works on top of `numpy` and is default in `pytorch`). If that is the case, and we find it is a hands-down winner over my own heuristic, then we can just have `evaluate()` pass *all* of the arguments directly to `opt_einsum`. This might also be left as a user choice.

Update: `opt_einsum` can now be enabled with the `tensorly` backend by the use of the following two lines, probably at the top-level:

```
1 tensorly_plugins.use_opt_einsum()
2 qcode.math.tensornet.backend_contract_path(True)
```

See also:

https://tensorly.org/stable/modules/generated/tensorly.plugins.use_opt_einsum.html#tensorly.plugins.use_opt_einsum

<https://optimized-einsum.readthedocs.io/en/stable/install.html#conda>

I'm not a big fan of how this works because, IMO, all decisions about how to contract tensors should be as local as possible. Say, for example, you wanted to merge two codes that had been using different backends for the tensors. It will be a lot more convenient if you don't have to force them to agree on one. This is why my tensors carry knowledge of their backends with them, and the only thing that matters when two tensors meet is that they are the same (which is presumably the case within any given code domain). Now there is this wart, where you insist that all tensors (even those with different backends) are either going to use the frontend or the backend contraction path. Probably this is ok for well over 90 cases, but I would like to clean it up. At the same time though, making the `backend_contract_path` option a feature of a backend is also not really (ie, could carry this option around with the tensors) because it is not a feature of the backend, it is a feature of a contraction within the context of a backend. We certainly do not want to insist that a tensor created with one way of determining the contraction path cannot be contracted with a tensor with a different "opinion" on the way of determining the contraction path. At the moment then, this is the least of evils. At least a user can dynamically switch back and forth.

Actually the obvious solution is between the extremes. Similar to timings, the interface class contains a function that can modify a module-level variable. So tensors with the same backend will all behave the same way, and this runtime distinction does not define a backend, but different backends can still behave differently (and you could have two nominally different backends if you want that still can communicate via raw tensors)